1. WPROWADZENIE

1.1. Wstęp

W dobie postępującej cyfryzacji usług i procesów biznesowych systemy informatyczne stanowią fundament operacyjnego funkcjonowania przedsiębiorstw z branży rozrywkowej [1]. Duże sieci kinowe dysponują zazwyczaj rozbudowanymi, dedykowanymi systemami klasy ERP (*Enterprise Resource Planning*) oraz zaawansowanymi platformami webowymi obsługującymi sprzedaż i dystrybucję biletów. Analiza rynku wskazuje jednak na istnienie wyraźnej luki technologicznej w segmencie kin lokalnych i studyjnych, które nie wymagają kosztownych, korporacyjnych rozwiązań chmurowych, lecz stabilnych, responsywnych i zintegrowanych systemów desktopowych typu POS (*Point of Sale*) [2]. Aplikacje okienkowe, pomimo systematycznie rosnącej popularności rozwiązań webowych, zachowują przewagę konkurencyjną w środowiskach stacjonarnej obsługi klienta, gdzie kluczowe znaczenie ma natychmiastowa responsywność interfejsu, bezpośredni dostęp do zasobów sprzętowych komputera (w tym elastyczna konfiguracja wielomonitorowa) oraz zdolność do pracy w trybie lokalnym, niezależnym od jakości łącza internetowego [3].

Niniejsza praca inżynierska stanowi odpowiedź na zidentyfikowane zapotrzebowanie. Opisuje ona proces projektowania i implementacji autorskiego systemu przeznaczonego do kompleksowej obsługi rezerwacji i sprzedaży biletów kinowych. Oprócz aspektu stricte użytkowego, istotną motywacją do realizacji projektu była chęć praktycznej weryfikacji paradygmatów współczesnej inżynierii oprogramowania. Projekt stanowi platformę do empirycznego zbadania możliwości technologii Windows Presentation Foundation (WPF) [4] oraz ewaluacji wzorca architektonicznego Model–View–ViewModel (MVVM) [5] w kontekście zapewnienia separacji odpowiedzialności, utrzymywalności kodu oraz skalowalności desktopowych aplikacji klasy *line-of-business* (LOB).

1.2. Cel główny i cele szczegółowe

Cel główny

Celem niniejszej pracy inżynierskiej jest zaprojektowanie oraz implementacja kompletnego systemu zarządzania rezerwacjami biletów kinowych na platformie .NET 10 [6] z wykorzystaniem technologii Windows Presentation Foundation (WPF). Praca ma na celu wykazanie, w jaki sposób połączenie wzorca architektonicznego MVVM, koncepcji wywodzących się z frameworka WAF (*WPF Application Framework*) [7] oraz nowoczesnych bibliotek wspomagających — Entity Framework Core [8], Community Toolkit MVVM [9] — pozwala na efektywne rozwiązywanie złożonych wyzwań inżynierskich występujących podczas budowy skalowalnych aplikacji desktopowych.

Cele szczegółowe

1. Analiza ekosystemu .NET 10 — badanie możliwości technologicznych języka C# [10], mechanizmów wstrzykiwania zależności (*Dependency Injection*) [11] oraz współczesnego środowiska uruchomieniowego platformy .NET.
2. Projektowanie warstwy prezentacji w WPF — analiza i praktyczne wykorzystanie zaawansowanych mechanizmów silnika renderującego opartego na DirectX, systemu wiązania danych (*Data Binding*) [12] oraz deklaratywnego opisu interfejsu w języku XAML (*Extensible Application Markup Language*) [13].
3. Implementacja architektury MVVM i wzorca Application Controller — praktyczne zastosowanie wzorca separacji warstw z wykorzystaniem generatorów kodu źródłowego

(*Source Generators*) biblioteki Community Toolkit MVVM [9] oraz wzorca kontrolera aplikacji (*Application Controller*) wywiedzionego z koncepcji frameworka WAF [7].

4. Budowa trwałej warstwy danych — implementacja modelu relacyjnego przy użyciu Entity Framework Core [8] w podejściu *Code-First*, zarządzanie migracjami schematu bazy danych oraz zapewnienie interoperacyjności z systemami zarządzania bazami danych PostgreSQL [14] i SQLite [15].
5. Dekompozycja i implementacja komponentów systemu — wytworzenie oraz szczegółowy opis struktury aplikacji obejmującej modele encji, klasy ViewModeli, zunifikowane widoki XAML oraz serwisy logiki biznesowej.
6. Rozszerzenie funkcjonalne systemu — integracja modułów dodatkowych, takich jak generator kodów QR (QRCoder) [16], system generowania dokumentów PDF (QuestPDF) [17], automatyzacja powiadomień poczty elektronicznej za pośrednictwem protokołu SMTP oraz obsługa konfiguracji wielomonitorowej.
7. Ewaluacja i weryfikacja — analiza spójności architektonicznej, testowanie przepływów biznesowych oraz ocena wydajności i bezpieczeństwa zaimplementowanych rozwiązań w kontekście wymagań systemu transakcyjnego.

1.3. Zakres pracy

Zakres niniejszej pracy inżynierskiej obejmuje pełny cykl życia oprogramowania (*Software Development Life Cycle*, SDLC) [18] dla aplikacji typu desktop — od fazy analizy wymagań i projektowania, poprzez implementację, aż po integrację zaawansowanych mechanizmów peryferyjnych. Wymiar teoretyczny pracy koncentruje się na krytycznym przeglądzie technologii z ekosystemu .NET oraz analizie architektonicznej wzorca MVVM w odniesieniu do kanonicznych zasad projektowania oprogramowania.

Wymiar praktyczny pracy, ściśle ograniczony do dostarczonego repozytorium projektu, obejmuje w szczególności:

- Zaprojektowanie i implementację warstwy bazodanowej. Stworzenie modelu relacyjnego z wykorzystaniem technologii Entity Framework Core w podejściu *Code-First* [8], dedykowanego pod silnik PostgreSQL z wykorzystaniem dostawcy Npgsql [19], z jednoczesnym zachowaniem wsparcia dla lekkiego środowiska deweloperskiego opartego o SQLite.
- Budowę logiki biznesowej i warstwy usług. Implementację mechanizmów rezerwacji biletów, zarządzania seansami oraz obsługi klientów. Zaprojektowanie bezpiecznej architektury z zastosowaniem wstrzykiwania zależności przez konstruktory (*Constructor Injection*) [11] oraz obsługi współbieżności w modelu optymistycznym (*Optimistic Concurrency Control*) [20].
- Opracowanie warstwy prezentacji (UI/UX). Stworzenie interfejsu użytkownika w technologii WPF i języku XAML, wykorzystującego bibliotekę CommunityToolkit.Mvvm do realizacji wiązania danych. Implementacja m.in. dynamicznych widoków sal kinowych oraz mechaniki obsługi stanowisk wyposażonych w wiele monitorów — z osobnym widokiem dla kasjera i klienta, współdzielącymi ten sam obiekt *MainViewModel* w celu synchronizacji stanu.
- Integrację funkcjonalności rozszerzonych. Wdrożenie modułów generowania biletów oraz raportów analitycznych w formacie PDF przy użyciu biblioteki QuestPDF [17], tworzenia kodów QR weryfikujących transakcje przy użyciu biblioteki QRCoder [16] oraz automatyzacji

wysyłki powiadomień e-mail za pośrednictwem wbudowanej klasy System.Net.Mail i protokołu SMTP.

Z zakresu pracy celowo wyłączono projektowanie zewnętrznej aplikacji mobilnej oraz webowej dla klienta końcowego, a także bezpośrednią integrację na poziomie sterowników z fizycznymi urządzeniami fiskalnymi, koncentrując się w całości na cyfrowej logice i interfejsie stanowiska operatora kina.

1.4. Założenia projektowe

Proces wytwarzania systemu został oparty na ściśle zdefiniowanych wymaganiach funkcjonalnych oraz niefunkcjonalnych, zgodnie z metodyką specyfikacji wymagań opisaną w standardzie IEEE 830 [21]. Do kluczowych założeń funkcjonalnych należą:

- Obsługa procesu sprzedaży i rezerwacji biletów z wykorzystaniem interaktywnej mapy miejsc, odzwierciedlającej ich aktualny status w czasie zbliżonym do rzeczywistego.
- Skalowalny mechanizm biletowy wspierający heterogeniczne typy biletów (normalny, ulgowy, studencki), kody rabatowe oraz automatycznie generowane vouchery promocyjne.
- Automatyczne generowanie personalizowanych biletów w formacie PDF, zawierających unikatowy kod QR, oraz ich dystrybucja na wskazany adres poczty elektronicznej klienta.
- Moduł analityczny umożliwiający wizualizację kluczowych wskaźników efektywności (KPI), generowanie wykresów trendów sprzedażowych oraz eksport zestawień do formatów PDF i CSV.
- Obsługa konfiguracji wielomonitorowej, polegająca na wyświetlaniu interaktywnego panelu podglądu miejsc na sali, obszaru reklamowego z dostępnymi seansami oraz harmonogramu — na dodatkowym ekranie skierowanym bezpośrednio do klienta.

Wśród wymagań niefunkcjonalnych główny nacisk położono na:

- Utrzymanie rygorystycznego podziału warstw aplikacji (separacja widoku od logiki biznesowej i logiki prezentacji) za sprawą konsekwentnego stosowania wzorca architektonicznego MVVM [5].
- Zapewnienie niezawodności i spójności transakcyjnej przy jednoczesnym dostępie wielu operatorów (kasjerów), poprzez zastosowanie mechanizmu optymistycznego blokowania współbieżności opartego na śledzeniu wersji rekordów (*RowVersion*) [20].
- Płynność działania interfejsu użytkownika, osiągniętą dzięki asynchronicznemu odpytywaniu bazy danych z wykorzystaniem wzorca TAP (*Task-based Asynchronous Pattern*) [22] oraz ładowaniu zasobów w wątkach tła.

1.5. Struktura pracy

Niniejsza praca została podzielona na cztery powiązane merytorycznie rozdziały, prowadzące czytelnika od koncepcji teoretycznej po gotowy produkt programistyczny:

- Rozdział 1 nakreśla kontekst projektu, definiuje jego cel, zakres oraz kluczowe wymagania funkcjonalne, niefunkcjonalne i założenia architektoniczne.
- Rozdział 2 zawiera systematyczny przegląd wykorzystanych technologii, frameworków oraz narzędzi, obejmujący architekturę platformy .NET, technologię WPF, wzorzec MVVM, system

zarządzania relacyjną bazą danych PostgreSQL oraz biblioteki wspomagające generowanie dokumentów PDF i kodów QR.

- Rozdział 3 stanowi główną, inżynierską część pracy. Szczegółowo dokumentuje proces implementacji systemu — począwszy od struktury modeli domenowych i warstwy dostępu do danych, poprzez logikę ViewModeli i mechanizmy kontroli współbieżności, aż po zaawansowane komponenty interfejsu użytkownika, moduły generowania raportów oraz obsługę środowiska wielomonitorowego.
- Rozdział 4 zawiera podsumowanie wykonanych prac, ewaluację stopnia osiągnięcia założonych celów oraz zarysowanie perspektyw dalszego rozwoju zbudowanego systemu.

---

2. PRZEGLĄD WYKORZYSTANYCH TECHNOLOGII I NARZĘDZI

2.1. Platforma .NET i język C#

Ewolucja ekosystemu do wytwarzania oprogramowania firmy Microsoft doprowadziła do powstania zunifikowanego, wieloplatformowego środowiska uruchomieniowego .NET, stanowiącego następcę zarówno klasycznego .NET Framework, jak i projektów .NET Core [6]. W ramach niniejszego projektu zdecydowano się na wykorzystanie platformy .NET w wersji 10, co potwierdza konfiguracja projektu określająca TargetFramework jako net10.0-windows. Wybór ten podyktowany był przede wszystkim dojrzałością platformy, wysoką wydajnością kompilatora JIT (*Just-In-Time*) nowej generacji (RyuJIT), bogatą biblioteką klas podstawowych (*Base Class Library*, BCL) oraz długoterminowym wsparciem producenta, co znacząco przyspiesza proces dostarczania funkcjonalności biznesowych [6].

Językiem programowania wykorzystanym do implementacji logiki systemu jest obiektowo zorientowany język C# [10], rozwijany przez Microsoft jako integralna część platformy .NET. Projekt intensywnie wykorzystuje jego zaawansowane mechanizmy językowe, w szczególności programowanie asynchroniczne z użyciem słów kluczowych async i await, realizujące wzorzec TAP (*Task-based Asynchronous Pattern*) [22]. Zastosowanie asynchroniczności jest kluczowe dla zachowania responsywności interfejsu użytkownika podczas operacji wejścia/wyjścia, takich jak odpytywanie bazy danych. Ponadto szeroko zastosowano mechanizm LINQ (*Language Integrated Query*) [23] do deklaratywnego i bezpiecznego typologicznie (*type-safe*) przetwarzania kolekcji danych w pamięci operacyjnej, co redukuje złożoność cyklomatyczną kodu i minimalizuje ryzyko błędów logicznych w warstwie transformacji danych.

2.2. Windows Presentation Foundation (WPF) jako technologia tworzenia aplikacji desktopowych

Windows Presentation Foundation (WPF) to zaawansowany framework prezentacyjny wchodzący w skład ekosystemu .NET, dedykowany do tworzenia bogatych interfejsów użytkownika (*Rich Client Applications*) dla systemu Windows [4]. W odróżnieniu od starszej technologii Windows Forms, opartej na rastrowym silniku GDI+, WPF wykorzystuje wektorowy silnik renderujący ze sprzętową akceleracją graficzną realizowaną za pośrednictwem biblioteki DirectX [3]. Architektura ta umożliwia płynne skalowanie interfejsu niezależnie od rozdzielczości ekranu, obsługę zaawansowanych animacji, transformacji oraz renderowanie grafiki 2D i 3D bez nadmiernego obciążania głównego wątku procesora — co w kontekście stacjonarnej aplikacji POS jest wymaganiem o krytycznym znaczeniu.

Kluczową innowacją architektoniczną wprowadzoną przez WPF jest radykalne oddzielenie warstwy wizualnej od logiki aplikacji za pomocą deklaratywnego języka znaczników XAML (*Extensible*

Application Markup Language) [13]. Model programistyczny WPF opiera się na rozbudowanym systemie właściwości zależności (*Dependency Properties*) [12] oraz zaawansowanym mechanizmie wiązania danych (*Data Binding*). Mechanizmy te umożliwiły w niniejszym projekcie stworzenie dynamicznych interfejsów — takich jak interaktywna mapa sali kinowej — których wygląd aktualizuje się automatycznie w odpowiedzi na zmiany stanu obiektów w warstwie logiki aplikacji, bez konieczności manualnego manipulowania elementami wizualnymi z poziomu kodu proceduralnego (*code-behind*).

2.3. Wzorzec architektoniczny Model–View–ViewModel (MVVM)

Model–View–ViewModel (MVVM) to wzorzec architektoniczny opracowany w firmie Microsoft przez Johna Gossma, stanowiący specjalizację wzorca prezentacyjnego PM (*Presentation Model*) Martina Fowlera, zoptymalizowaną pod kątem technologii wspierających dwukierunkowe wiązanie danych — w szczególności WPF i Silverlight [5]. Nadrzędnym celem MVVM jest osiągnięcie całkowitej separacji interfejsu użytkownika od logiki biznesowej i logiki prezentacji, co w konsekwencji zwiększa testowalność jednostkową, utrzymywalność oraz czytelność kodu źródłowego [24].

Wzorzec ten dekomponuje aplikację na trzy współpracujące ze sobą warstwy:

1. Model (*Model*): Reprezentuje warstwę domeny i logiki biznesowej. W kontekście niniejszego projektu są to klasy encji odzwierciedlające rzeczywiste byty dziedziny problemowej (np. Film, Screening, Ticket, Reservation), które hermetyzują dane oraz podstawowe reguły walidacji, nie posiadając żadnej wiedzy o sposobie ich prezentacji w interfejsie użytkownika.

2. Widok (*View*): Odpowiada za strukturę wizualną, układ i wygląd interfejsu. W aplikacji WPF widok jest definiowany deklaratywnie w plikach XAML (np. LoginView.xaml, SeatSelectionView.xaml). Jego jedynym zadaniem jest prezentowanie danych dostarczonych przez ViewModel oraz przekazywanie interakcji użytkownika (kliknięć, gestów) w formie komend (*Commands*). Zgodnie z zasadami wzorca, widok nie powinien zawierać logiki biznesowej w pliku *code-behind*.

3. ViewModel (*Model Widoku*): Pełni rolę pośrednika (*mediator*) pomiędzy Modelem a Widokiem. Transformuje dane z Modelu na format przystępny dla Widoku, implementuje interfejs INotifyPropertyChanged odpowiedzialny za powiadamianie widoku o zmianach stanu właściwości, a także udostępnia publiczne operacje w postaci obiektów implementujących interfejs ICommand, wywoływanych w odpowiedzi na akcje użytkownika [5].

Zastosowanie wzorca MVVM w systemie rezerwacji biletów skutkuje wysoką modułowością architektury, co umożliwiło m.in. niezależne projektowanie i implementowanie poszczególnych ekranów aplikacji — ekranu kasjera (MainWindow) i ekranu klienta (CustomerWindow) — które współdzielą tę samą instancję MainViewModel oraz wspólne serwisy logiki biznesowej.

Rysunek 1. Przepływ danych we wzorcu architektonicznym MVVM. Opracowanie własne.

2.4. Framework WAF oraz Community Toolkit MVVM

Implementacja wzorca MVVM w czystym języku C# wiąże się z koniecznością tworzenia znacznej ilości powtarzalnego kodu szablonowego (*boilerplate code*), w szczególności związanego z ręczną implementacją interfejsu INotifyPropertyChanged oraz komend realizujących interfejs ICommand [24]. W celu optymalizacji tego procesu i redukcji objętości klas ViewModeli, w projekcie wykorzystano nowoczesne narzędzia wspierające architekturę MVVM.

Głównym instrumentem wspomagającym warstwę logiki prezentacji jest biblioteka CommunityToolkit.Mvvm (w wersji 8.4.0), rozwijana i utrzymywana przez Microsoft w ramach inicjatywy .NET Community Toolkit [9]. Jej zasadniczą zaletą jest wykorzystanie mechanizmu generatorów kodu źródłowego (*Source Generators*) [25] wprowadzonych w platformie .NET. Dzięki zastosowaniu deklaratywnych atrybutów — takich jak [ObservableProperty] czy [RelayCommand] — kompilator automatycznie generuje wymagany kod implementacyjny w trakcie kompilacji (*compile-time code generation*). Technika ta drastycznie redukuje objętość klas ViewModeli, zwiększa ich czytelność, eliminuje typowe błędy programistyczne związane z ręczną notyfikacją zmian właściwości i ułatwia długoterminowe utrzymanie bazy kodu.

Ponadto architekturę aplikacji ustrukturyzowano, czerpiąc inspirację z założeń WPF Application Framework (WAF) [7]. Koncepcje wywodzące się z WAF posłużyły do implementacji wzorca *Application Controller* [26], który w omawianym projekcie centralizuje zarządzanie nawigacją pomiędzy poszczególnymi widokami (np. przełączanie między ekranem logowania a panelem głównym) oraz zarządza cyklem życia aplikacji desktopowej. Rolę orkiestratora pełni w tym przypadku klasa MainViewModel, koordynująca komunikację między ViewModelami za pośrednictwem zdarzeń (*event-driven communication*), takich jak LoginSuccess czy LogoutRequested.

2.5. System zarządzania bazą danych PostgreSQL

Wybór odpowiedniego systemu zarządzania bazą danych (SZBD) ma krytyczne znaczenie dla aplikacji transakcyjnych, takich jak systemy rezerwacji biletów, w których wymagana jest ścisła spójność danych gwarantowana przez właściwości ACID (*Atomicity, Consistency, Isolation, Durability*) [27]. Jako główne, produkcyjne środowisko bazodanowe w projekcie wybrano relacyjny system PostgreSQL [14].

PostgreSQL to zaawansowany, otwartoźródłowy (*open-source*) obiektowo-relacyjny system zarządzania bazą danych, charakteryzujący się wysoką wydajnością, niezawodnością oraz rozbudowanymi mechanizmami kontroli współbieżności opartymi na architekturze MVCC (*Multi-Version Concurrency Control*) [14]. W kontekście systemu kinowego, w którym wielu operatorów (kasjerów) może w tym samym ułamku sekundy podejmować próbę rezerwacji tego samego miejsca na sali, stabilny silnik relacyjny z gwarancjami transakcyjnymi jest niezbędny do zapobiegania anomaliom danych, takim jak *dirty reads*, *phantom reads* czy utrata aktualizacji (*lost updates*).

Architektura projektu zakłada jednocześnie elastyczność środowiska uruchomieniowego — dzięki zastosowaniu abstrakcji na poziomie warstwy ORM, system wspiera również lekką bazę SQLite [15], wykorzystywaną w roli środowiska deweloperskiego i testowego, co przyspiesza cykl iteracyjny podczas wytwarzania oprogramowania.

2.6. Entity Framework Core jako warstwa dostępu do danych

Do komunikacji pomiędzy obiektowym kodem aplikacji napisanym w C# a relacyjną bazą danych (PostgreSQL/SQLite) wykorzystano Entity Framework Core (EF Core) [8]. Jest to nowoczesny, wieloplatformowy framework mapowania obiektowo-relacyjnego (ORM — *Object-Relational Mapping*) rozwijany przez Microsoft, stanowiący de facto standardowy mechanizm dostępu do danych w ekosystemie .NET.

W projekcie konsekwentnie zastosowano podejście Code-First [28]. Oznacza to, że schemat bazy danych — tabele, relacje, klucze obce, indeksy — nie był projektowany ręcznie w języku SQL, lecz został wygenerowany automatycznie na podstawie modeli domenowych (klas C#, takich jak Film, Screening, Ticket). Wszelkie modyfikacje struktury modeli są propagowane do bazy danych za

pomocą mechanizmu systematycznych migracji (*EF Core Migrations*) [29]. Centralnym elementem warstwy danych jest klasa CinemaDbContext, dziedzicząca po klasie bazowej DbContext, która konfiguruje zbiory encji (DbSet<T>) dla poszczególnych bytów domenowych.

Kluczowym aspektem inżynieryjnym zastosowanym w warstwie persystencji jest obsługa optymistycznego blokowania współbieżności (*Optimistic Concurrency Control*) [20]. W przeciwieństwie do strategii pesymistycznej, która fizycznie blokuje rekordy w bazie podczas ich odczytu (co drastycznie obniżałoby przepustowość systemu POS przy wielu stanowiskach kasjerskich), EF Core wykorzystuje specjalne tokeny współbieżności (*concurrency tokens*). W omawianym projekcie mechanizm ten jest realizowany za pośrednictwem właściwości RowVersion — automatycznie inkrementowanej przy każdej modyfikacji rekordu. Jeśli dwóch operatorów podejmie próbę jednoczesnej aktualizacji tego samego zasobu, EF Core zgłosi wyjątek DbUpdateConcurrencyException dla transakcji, która jako druga podejmie próbę zapisu [20]. Mechanizm ten umożliwił bezpieczną implementację procesu rezerwacji biletów, eliminując ryzyko podwójnej sprzedaży tego samego miejsca (*double-booking*) — jednego z najpoważniejszych defektów w systemach rezerwacyjnych.

Bibliografia

[1] P. Beynon-Davies, *Systemy informacyjne. Wprowadzenie*, Warszawa: Wydawnictwa Naukowo-Techniczne, 2004.

[2] R. S. Pressman, B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., New York: McGraw-Hill Education, 2019.

[3] Microsoft, „WPF overview," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/

[4] Microsoft, „Windows Presentation Foundation documentation," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/

[5] Microsoft, „The MVVM pattern," .NET Application Architecture Guide, Microsoft Learn, 2023. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm

[6] Microsoft, „What is .NET?," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/core/introduction

[7] jbe2277, „WPF Application Framework (WAF)," GitHub Repository, 2024. [Online]. Dostępne: https://github.com/jbe2277/waf

[8] Microsoft, „Entity Framework Core overview," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/

[9] Microsoft, „MVVM Toolkit introduction," .NET Community Toolkit, Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/

[10] Microsoft, „A tour of the C# language," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/

[11] Microsoft, „Dependency injection in .NET," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection

[12] Microsoft, „Data binding overview – WPF," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/data/data-binding-overview

[13] Microsoft, „XAML overview (WPF .NET)," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/xaml/

[14] The PostgreSQL Global Development Group, „PostgreSQL: The World's Most Advanced Open Source Relational Database – Documentation," 2024. [Online]. Dostępne: https://www.postgresql.org/docs/

[15] SQLite Consortium, „SQLite Documentation," 2024. [Online]. Dostępne: https://www.sqlite.org/docs.html

[16] C. Mecklenbrauck, „QRCoder – QR code generator library for .NET," GitHub Repository, 2024. [Online]. Dostępne: https://github.com/codebude/QRCoder

[17] QuestPDF Contributors, „QuestPDF – Open Source .NET Library for PDF Document Generation," 2024. [Online]. Dostępne: https://www.questpdf.com/

[18] I. Sommerville, *Software Engineering*, 10th ed., Harlow: Pearson, 2015.

[19] Npgsql Development Team, „Npgsql – .NET Access to PostgreSQL," 2024. [Online]. Dostępne: https://www.npgsql.org/

[20] Microsoft, „Handling concurrency conflicts – EF Core," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/saving/concurrency

[21] IEEE, *IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications*, New York: IEEE, 1998.

[22] Microsoft, „Task-based asynchronous pattern (TAP)," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/standard/asynchronous-programming-patterns/task-based-asynchronous-pattern-tap

[23] Microsoft, „Language Integrated Query (LINQ)," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/csharp/linq/

[24] M. Fowler, *Patterns of Enterprise Application Architecture*, Boston: Addison-Wesley Professional, 2002.

[25] Microsoft, „Source Generators," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/source-generators-overview

[26] M. Fowler, „Application Controller," w *Patterns of Enterprise Application Architecture*, 2002. [Online]. Dostępne: https://martinfowler.com/eaaCatalog/applicationController.html

[27] A. Silberschatz, H. F. Korth, S. Sudarshan, *Database System Concepts*, 7th ed., New York: McGraw-Hill Education, 2019.

[28] Microsoft, „Code First to a New Database – EF Core," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/modeling/

[29] Microsoft, „Migrations Overview – EF Core," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/

2.7. Dodatkowe biblioteki i mechanizmy wykorzystane w projekcie

Realizacja kompletnego systemu biletowego klasy *Point of Sale* wymaga wyjścia poza standardowe operacje bazodanowe i konstrukcję interfejsu użytkownika. Aby spełnić zdefiniowane w podrozdziale 1.4 wymagania funkcjonalne — w szczególności te dotyczące obsługi posprzedażowej klienta oraz generowania materiałów raportowych — architekturę systemu rozszerzono o wyspecjalizowane biblioteki zewnętrzne oraz natywne mechanizmy komunikacji sieciowej platformy .NET. Niniejszy podrozdział prezentuje charakterystykę tych komponentów w kontekście ich roli architektonicznej w systemie.

2.7.1. QuestPDF — generowanie dokumentów PDF

Jednym z kluczowych wymogów funkcjonalnych systemu była zdolność do natychmiastowego generowania elektronicznych biletów kinowych oraz zestawień analitycznych w przenośnym formacie dokumentów PDF (*Portable Document Format*). W tym celu zintegrowano bibliotekę **QuestPDF** w wersji 2025.12.1, dystrybuowaną na licencji *Community* [1]. QuestPDF jest nowoczesnym, otwartoźródłowym narzędziem dla ekosystemu .NET, zoptymalizowanym pod kątem wysokiej wydajności generowania dokumentów programistycznie — bez udziału silników przeglądarkowych czy komponentów pośredniczących.

Z perspektywy inżynieryjnej wybór QuestPDF podyktowany był jego architekturą opartą na tzw. **płynnym interfejsie programistycznym** (*Fluent API*) [2]. W odróżnieniu od tradycyjnych narzędzi do generowania dokumentów PDF, które opierają się na konwersji znaczników HTML do formatu docelowego (np. za pośrednictwem silnika Chromium) lub wykorzystują niskopoziomowe API manipulujące strumieniami binarnymi, QuestPDF stosuje podejście deklaratywne [1]. Kompozycja dokumentu — układ strony, tabele, nagłówki, stopki — jest definiowana bezpośrednio w kodzie C# z zachowaniem pełnej kontroli typów w trakcie kompilacji (*compile-time type safety*). Takie podejście jest zgodne z paradygmatem programowania deklaratywnego, który redukuje liczbę defektów wynikających z niespójności pomiędzy szablonem dokumentu a danymi źródłowymi [3].

W omawianym projekcie biblioteka QuestPDF odpowiada za dwa odrębne procesy dokumentowe, zrealizowane w postaci dedykowanych klas serwisowych:

- **Generowanie spersonalizowanych biletów** — klasa TicketPdfGenerator (plik Services/Pdf/TicketPdfGenerator.cs) dynamicznie komponuje dokument formatu A4 zawierający: nagłówek z logo systemu i kodem QR transakcji, sekcję informacyjną z danymi seansu (tytuł filmu, data, godzina, sala), tabelaryczne zestawienie zarezerwowanych miejsc z cenami, a także — w przypadku zakupu co najmniej dwóch biletów — automatycznie generowany voucher barowy [4].
- **Eksport raportów analitycznych** — klasa PdfReportService (plik Services/Pdf/PdfReportService.cs) generuje wielostronicowe raporty w formacie A4 *landscape*, zawierające karty KPI (*Key Performance Indicators*), zestawienia sprzedaży w ujęciu dziennym, tygodniowym oraz w przekroju sal kinowych i gatunków filmowych [4].

2.7.2. QRCoder — generowanie kodów QR

W celu zapewnienia mechanizmu weryfikacji autentyczności biletów oraz umożliwienia szybkiej walidacji transakcji przed wejściem na salę kinową, system wyposażono w moduł generowania dwuwymiarowych kodów QR (*Quick Response*) [5]. Do realizacji tej funkcjonalności wykorzystano bibliotekę **QRCoder** w wersji 1.7.0 — w pełni zarządzany (*fully managed*) generator kodów QR, napisany wyłącznie w języku C#, niewymagający zależności od bibliotek natywnych [6].

Standard QR Code, zdefiniowany w normie ISO/IEC 18004:2015 [5], określa strukturę symbolu dwuwymiarowego zdolnego do kodowania danych alfanumerycznych z wbudowanym mechanizmem korekcji błędów (*Error Correction*). Biblioteka QRCoder implementuje cztery poziomy korekcji zdefiniowane w normie — L (7%), M (15%), Q (25%) i H (30%) [6]. W omawianym projekcie zastosowano poziom **Q** (25% redundancji korekcyjnej), co stanowi kompromis pomiędzy gęstością kodowania a odpornością na częściowe uszkodzenie wydruku.

Mechanika działania modułu — zaimplementowana w prywatnej metodzie GenerateQrCode klasy TicketPdfGenerator — polega na przekształceniu unikatowego identyfikatora transakcji (np. ciągu znaków w formacie TKT-20260315-000042 przypisanego do konkretnego biletu w bazie danych) w macierz pikseli reprezentującą symbol QR. Klasa PngByteQRCode generuje obraz w formacie PNG jako tablicę bajtów (byte[]), która następnie jest bezpośrednio osadzana jako strumień danych binarnych w dokumencie PDF konstruowanym przez QuestPDF — za pośrednictwem metody .Image(qrBytes). Takie podejście, realizujące wzorzec *in-memory processing*, eliminuje konieczność zapisywania tymczasowych plików graficznych na dysku twardym komputera kasowego, minimalizując liczbę operacji wejścia/wyjścia (*I/O*) i przyspieszając proces finalizacji transakcji [7].

2.7.3. Mechanizm wysyłania powiadomień e-mail

Ostatnim etapem przepływu rezerwacyjnego jest dostarczenie wygenerowanego biletu PDF do klienta drogą elektroniczną. Logikę tę zrealizowano z wykorzystaniem klas wbudowanych w przestrzeń nazw System.Net.Mail platformy .NET, implementujących obsługę protokołu **SMTP** (*Simple Mail Transfer Protocol*), zdefiniowanego w specyfikacji RFC 5321 [8]. Warstwa sieciowa przesyłu wiadomości została hermetyzowana w dedykowanej klasie EmailService (plik Services/EmailService.cs).

Klasa EmailService realizuje następujący model operacyjny: konstruuje obiekt MailMessage zawierający spersonalizowaną treść powiadomienia oraz załącznik w postaci pliku PDF biletu, a następnie przekazuje go do obiektu SmtpClient skonfigurowanego do komunikacji z zewnętrznym serwerem pocztowym (w bieżącej implementacji jest to serwer smtp.gmail.com na porcie 587) [9]. Połączenie z serwerem SMTP zabezpieczone jest za pośrednictwem protokołu **TLS** (*Transport Layer Security*), aktywowanego poprzez właściwość EnableSsl = true, co gwarantuje poufność i integralność przesyłanych danych zgodnie z zaleceniami RFC 8314 dotyczącymi stosowania TLS w protokołach pocztowych [10]. Uwierzytelnianie realizowane jest z wykorzystaniem mechanizmu *App Passwords* usługi Gmail, co eliminuje konieczność ujawniania głównego hasła konta nadawcy [11].

Z perspektywy architektury aplikacji desktopowej, w której priorytetem jest zachowanie płynności interfejsu użytkownika, należy podkreślić, że proces wysyłki wiadomości e-mail — jako operacja sieciowa o potencjalnie wysokiej latencji — powinien być realizowany asynchronicznie, w wątku tła, zgodnie ze wzorcem TAP (*Task-based Asynchronous Pattern*) [12]. Takie podejście zapobiega blokowaniu głównego wątku dispatera WPF (*UI thread*) w czasie oczekiwania na odpowiedź od zewnętrznego serwera pocztowego, eliminując ryzyko zamrożenia interfejsu z perspektywy operatora kasy.

Bibliografia (kontynuacja numeracji z poprzednich rozdziałów)

[1] QuestPDF Contributors, „QuestPDF — Open Source .NET Library for PDF Document Generation," 2024. [Online]. Dostępne: https://www.questpdf.com/

[2] M. Fowler, „FluentInterface," martinfowler.com, 2005. [Online]. Dostępne: https://martinfowler.com/bliki/FluentInterface.html

[3] R. S. Pressman, B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., New York: McGraw-Hill Education, 2019.

[4] Repozytorium projektu — pliki Services/Pdf/TicketPdfGenerator.cs oraz Services/Pdf/PdfReportService.cs. [Online]. Dostępne: https://github.com/1Stazy/CA_TEST/tree/main/Services/Pdf

[5] ISO/IEC, „ISO/IEC 18004:2015 — Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification," International Organization for Standardization, 2015.

[6] C. Mecklenbrauck, „QRCoder — QR code generator library for .NET," GitHub Repository, 2024. [Online]. Dostępne: https://github.com/codebude/QRCoder

[7] Microsoft, „Memory<T> and Span<T> usage guidelines," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/standard/memory-and-spans/memory-t-usage-guidelines

[8] J. Klensin, „RFC 5321 — Simple Mail Transfer Protocol," Internet Engineering Task Force (IETF), 2008. [Online]. Dostępne: https://datatracker.ietf.org/doc/html/rfc5321

[9] Microsoft, „SmtpClient Class," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/api/system.net.mail.smtpclient

[10] K. Moore, C. Newman, „RFC 8314 — Cleartext Considered Obsolete: Use of Transport Layer Security (TLS) for Email Submission and Access," Internet Engineering Task Force (IETF), 2018. [Online]. Dostępne: https://datatracker.ietf.org/doc/html/rfc8314

[11] Google, „Logowanie za pomocą haseł aplikacji," Pomoc konta Google, 2024. [Online]. Dostępne: https://support.google.com/accounts/answer/185833

[12] Microsoft, „Task-based asynchronous pattern (TAP)," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/standard/asynchronous-programming-patterns/task-based-asynchronous-pattern-tap

---

3. IMPLEMENTACJA APLIKACJI DO REZERWACJI BILETÓW

3.1. Analiza wymagań funkcjonalnych i niefunkcjonalnych systemu

Proces implementacji systemu zarządzania rezerwacjami biletów kinowych poprzedzono szczegółową analizą wymagań, której struktura została oparta na metodyce specyfikacji wymagań zalecanej przez standard IEEE 830 [1]. Wymagania podzielono na dwie zasadnicze kategorie: wymagania funkcjonalne, definiujące oczekiwane zachowanie systemu (*behavioral requirements*), oraz wymagania niefunkcjonalne, określające atrybuty jakościowe oprogramowania (*quality attributes*) [2].

Wymagania funkcjonalne

Wśród krytycznych wymagań funkcjonalnych zdefiniowano:

1. **Zarządzanie autoryzacją i rolami użytkowników.** System musi implementować mechanizm kontroli dostępu oparty na rolach (*Role-Based Access Control*, RBAC) [3], rozróżniając co najmniej dwa poziomy uprawnień: użytkownika o uprawnieniach administracyjnych (pełny dostęp do katalogu filmów, zarządzania harmonogramami i modułu raportowego) oraz kasjera (dostęp ograniczony do modułu sprzedaży i obsługi rezerwacji). Realizację tego wymagania wspiera klasa AuthenticationService (plik Services/AuthenticationService.cs) odpowiedzialna za walidację poświadczeń oraz model encji User przechowujący identyfikator roli.

2. **Obsługę procesu sprzedaży i rezerwacji biletów.** Wymagane jest dostarczenie interaktywnego, graficznego widoku sali kinowej umożliwiającego zaznaczanie miejsc — w tym zaznaczanie prostokątne (*rectangular selection*) i metodą przeciągnięcia (*drag-to-select*) — z odzwierciedleniem aktualnego statusu dostępności w czasie zbliżonym do rzeczywistego. Wymaganie to jest realizowane przez klasę SeatSelectionViewModel w połączeniu z odpowiadającym jej widokiem SeatSelectionView.xaml.

3. **Zarządzanie cennikiem i promocjami.** System musi wspierać heterogeniczne typy biletów (normalny, ulgowy, studencki) o dynamicznie konfigurowalnych cenach, obsługę kodów rabatowych (encja PromoCode) oraz automatyczne generowanie voucherów promocyjnych — np. darmowy zestaw barowy przy zakupie co najmniej dwóch biletów. Logika cenowa i promocyjna jest hermetyzowana w klasie TicketSummaryViewModel.

4. **Moduł raportowy i analityczny.** Aplikacja musi generować zaawansowane raporty w formacie PDF (za pośrednictwem klas PdfReportService i TicketPdfGenerator), prezentować graficzne wykresy sprzedaży oraz kluczowe wskaźniki efektywności (KPI — *Key Performance Indicators*) [4], takie jak łączny przychód, średnia cena biletu, wskaźnik obłożenia sal czy rozkład sprzedaży w przekroju dni tygodnia i godzin. Dane analityczne są przetwarzane i eksponowane przez klasę ReportsViewModel.

5. **Rozszerzoną obsługę klienta.** System musi wspierać konfigurację wielomonitorową (dedykowany ekran podglądu dla klienta), generowanie biletów z osadzonymi kodami QR weryfikującymi transakcje oraz automatyczną wysyłkę potwierdzeń na adres poczty elektronicznej klienta za pośrednictwem serwisu EmailService.

Wymagania niefunkcjonalne

Do kluczowych wymagań niefunkcjonalnych zaliczono:

- **Środowisko uruchomieniowe:** System operacyjny Windows 10 lub nowszy; platforma .NET 10 z deklaracją TargetFramework ustaloną na net10.0-windows [5].

- **Architektura:** Rygorystyczny podział na warstwy zgodny z paradygmatem MVVM (*Model–View–ViewModel*) [6], zapewniający separację logiki biznesowej od logiki prezentacji i warstwy wizualnej.

- **Baza danych:** Interoperacyjność z produkcyjnym środowiskiem PostgreSQL [7] przy jednoczesnym wsparciu dla lekkiej bazy SQLite [8] w fazie deweloperskiej, realizowana poprzez abstrakcję dostawcy (*database provider*) frameworka Entity Framework Core [9].

- **Interfejs użytkownika:** Responsywny interfejs graficzny wykonany w technologii WPF [10], wspierający tryb ciemny (*dark theme*) w module raportowym, zrealizowany z wykorzystaniem dedykowanych konwerterów wartości (*Value Converters*) [11] — takich jak

BooleanToColorConverter, BooleanToVisibilityConverter czy IsFutureDateConverter — rejestrowanych jako globalne zasoby aplikacji w pliku App.xaml.

3.2. Architektura aplikacji

Aplikację zaprojektowano w oparciu o **architekturę wielowarstwową** (*Layered Architecture*) [12], w której centralnym wzorcem organizującym strukturę kodu źródłowego jest MVVM [6]. Strukturę logiczną systemu podzielono na cztery współpracujące ze sobą warstwy, komunikujące się w trybie asynchronicznym z wykorzystaniem wzorca TAP (*Task-based Asynchronous Pattern*) [13].

1. Warstwa prezentacji (WPF/XAML)

Stanowi punkt wejścia dla interakcji z użytkownikiem. Składa się z **13 widoków deklaratywnych** w formacie XAML, w tym m.in.: LoginView, DashboardView, MoviesView, MovieDetailView, SeatSelectionView, TicketSummaryView, TicketManagementView, ScheduleView, ReportsView, BookingConfirmationWindow oraz CustomMessageBox. Dodatkowo warstwa ta obejmuje podkatalog Views/Controls z reużywalnymi komponentami interfejsu. Logika widoków (*code-behind*) została konsekwentnie zredukowana do minimum — w zgodzie z zasadami wzorca MVVM cała komunikacja pomiędzy widokiem a logiką prezentacji odbywa się za pośrednictwem mechanizmu wiązania danych (*Data Binding*) [11] i komend (*Commands*).

Charakterystyczną cechą architektoniczną aplikacji jest **dualny system okien**: główne okno kasjera (MainWindow) oraz dedykowane okno klienta (CustomerWindow). Oba okna współdzielą tę samą instancję MainViewModel jako kontekst danych (*DataContext*), co zapewnia automatyczną synchronizację stanu między stanowiskiem operatora a ekranem ekspozycji klienta [14].

2. Warstwa logiki prezentacji (ViewModels)

Oparta na bibliotece CommunityToolkit.Mvvm [15], warstwa ta obejmuje **12 klas ViewModeli**: MainViewModel (orkiestrator), LoginViewModel, DashboardViewModel, MoviesViewModel, MovieDetailViewModel, SeatSelectionViewModel, TicketSummaryViewModel, TicketManagementViewModel, ScheduleViewModel, ReportsViewModel, ClientScheduleViewModel oraz klasę pomocniczą GenreItem. Wszystkie klasy ViewModeli dziedziczą po wspólnej klasie bazowej ViewModelBase, co realizuje wzorzec *Layer Supertype* opisany przez Fowlera [16]. Warstwa ta przechwytuje akcje użytkownika zarejestrowane za pomocą atrybutu [RelayCommand] i aktualizuje właściwości powiązane z widokiem poprzez atrybut [ObservableProperty], generując wymagany kod notyfikacyjny w trakcie kompilacji (*compile-time source generation*) [15].

Klasa MainViewModel pełni rolę centralnego **kontrolera aplikacji** (*Application Controller*) [17] — zarządza nawigacją pomiędzy widokami poprzez właściwości CurrentView (ekran kasjera) i ClientView (ekran klienta), subskrybuje zdarzenia z podrzędnych ViewModeli (np. LoginSuccess, LogoutRequested, ShowClientScheduleRequested) oraz koordynuje cykl życia aplikacji, w tym automatyczną rotację treści na ekranie klienta realizowaną za pośrednictwem obiektu DispatcherTimer.

3. Warstwa usług (Services)

Hermetyzuje złożoną logikę biznesową i operacje wejścia/wyjścia (*I/O*). W projekcie zaimplementowano dedykowane serwisy w następującej strukturze:

Serwis	Plik źródłowy	Odpowiedzialność
AuthenticationService	Services/AuthenticationService.cs	Walidacja poświadczeń i uwierzytelnianie użytkowników
CinemaDbContext	Services/CinemaDbContext.cs	Kontekst ORM — konfiguracja zbiorów encji i dostawcy bazy danych
DatabaseGenerator	Services/DatabaseGenerator.cs	Inicjalizacja schematu bazy i generowanie danych startowych
DataSeeder	Services/DataSeeder.cs	Wypełnianie bazy danych danymi testowymi
EmailService	Services/EmailService.cs	Wysyłka powiadomień e-mail z załącznikami PDF via SMTP
ScreenManager	Services/ScreenManager.cs	Zarządzanie konfiguracją wielomonitorową
ReportSettingsService	Services/ReportSettingsService.cs	Konfiguracja parametrów modułu raportowego
TicketPdfGenerator	Services/Pdf/TicketPdfGenerator.cs	Generowanie biletów PDF z kodami QR
PdfReportService	Services/Pdf/PdfReportService.cs	Generowanie raportów analitycznych PDF
Konwertery wartości	Services/Converters/*.cs	Transformacja danych na potrzeby wiązania danych w XAML

Serwisy są instancjonowane i przekazywane do klas ViewModeli, co ułatwia zarządzanie zależnościami i umożliwia przyszłe wprowadzenie formalnego kontenera DI (*Dependency Injection Container*) [18].

4. Warstwa domenowa i dostępu do danych (Models & Data)

Zawiera definicje **11 encji** odzwierciedlających model relacyjny bazy danych: Film, Director, Hall, Seat, Screening, Reservation, Ticket, TicketTypeDefinition, PromoCode, Advertisement oraz User. Centralnym elementem warstwy persystencji jest klasa CinemaDbContext, dziedzicząca po DbContext frameworka Entity Framework Core [9], konfigurująca zbiory encji (DbSet<T>) oraz ich relacje.

Punkt startowy aplikacji

Przepływ sterowania w aplikacji rozpoczyna się w plikach App.xaml oraz App.xaml.cs, które definiują globalne zasoby (*Application Resources*), konwertery wartości (m.in. BooleanToColorConverter, BooleanToVisibilityConverter, IsFutureDateConverter) oraz stanowią główny punkt kompozycji aplikacji (*Composition Root*) [19]. W metodzie OnStartup następuje: (1) konfiguracja kultury pl-PL dla całej aplikacji, (2) inicjalizacja bazy danych za pośrednictwem DatabaseGenerator.GenerateFinalDatabase(), (3) instancjonowanie współdzielonego MainViewModel, (4) utworzenie i wyświetlenie okna kasjera (MainWindow), a następnie (5)

utworzenie okna klienta (CustomerWindow) i przeniesienie go na drugi monitor za pośrednictwem metody ScreenManager.OpenCustomerWindow().

3.3. Projekt bazy danych

Moduł persystencji danych w aplikacji został zrealizowany w podejściu **Code-First** przy użyciu Entity Framework Core [9]. Oznacza to, że relacyjny schemat bazy danych — tabele, kolumny, relacje, klucze obce, indeksy — został wygenerowany automatycznie na podstawie definicji klas obiektowych (encji) napisanych w języku C#, a nie projektowany ręcznie w języku SQL. Taka koncepcja umożliwia pełną kontrolę nad wersjonowaniem schematu bazy danych za pośrednictwem mechanizmu migracji (*EF Core Migrations*) [20], co jest praktyką rekomendowaną w kontekście ciągłej integracji i dostarczania oprogramowania (*CI/CD*) [2].

System wspiera dwa środowiska bazodanowe: **SQLite** [8] — wykorzystywane we wczesnych fazach deweloperskich i testowych, charakteryzujące się zerową konfiguracją i przechowywaniem danych w pojedynczym pliku — oraz docelowe, produkcyjne środowisko oparte na relacyjnym systemie **PostgreSQL** [7] z dostawcą Npgsql [21]. Zmiana dostawcy bazy danych realizowana jest w klasie kontekstu CinemaDbContext (plik Services/CinemaDbContext.cs), która dziedziczy po wbudowanej klasie DbContext i konfiguruje łańcuch połączenia (*connection string*) oraz odpowiedni dostawca za pośrednictwem metody OnConfiguring.

Model domenowy

Zgodnie z analizą repozytorium, model domenowy składa się z **11 encji** powiązanych ze sobą kluczami obcymi w strukturze relacyjnej. Do najistotniejszych z nich należą:

- **Film** — przechowuje informacje katalogowe o produkcjach kinowych (tytuł, czas trwania, gatunek, opis, ścieżka do plakatu). Powiązany relacją wiele-do-jednego z encją Director.
- **Director** — reprezentuje reżysera filmowego. Jedna instancja reżysera może być powiązana z wieloma filmami (relacja jeden-do-wielu).
- **Hall** — modeluje fizyczną salę kinową. Powiązana relacją jeden-do-wielu z encją Seat (kolekcja miejsc w sali) oraz z encją Screening (harmonogram seansów przypisanych do sali).
- **Seat** — reprezentuje pojedyncze miejsce w sali kinowej, identyfikowane przez numer rzędu (*Row*) i numer miejsca (*Number*). Powiązane kluczem obcym z encją Hall.
- **Screening** (Seans) — stanowi centralny byt procesu sprzedażowego. Łączy dany film z określoną salą kinową oraz konkretnym terminem projekcji (*Start*). Powiązane relacjami wiele-do-jednego z encjami Film i Hall.
- **Reservation** (Rezerwacja) — reprezentuje transakcję rezerwacji dokonywaną przez klienta, obejmującą dane identyfikacyjne klienta (CustomerName, CustomerEmail), status (Active, Cancelled) oraz datę utworzenia. Powiązana relacją jeden-do-wielu z encją Ticket.
- **Ticket** (Bilet) — reprezentuje pojedynczy bilet przypisany do konkretnego miejsca na konkretnym seansie. Przechowuje numer rzędu, numer miejsca, cenę zapłaconą, typ biletu, numer transakcji (*TransactionNumber*) oraz status. Powiązany kluczem obcym z encją Reservation.
- **TicketTypeDefinition** — definiuje dostępne typy biletów (normalny, ulgowy, studencki) wraz z przypisanymi cenami i statusem aktywności, umożliwiając dynamiczną konfigurację cennika bez modyfikacji kodu źródłowego.

- **PromoCode** — modeluje kody promocyjne z określoną wartością rabatu (kwotową lub procentową) i flagą aktywności.
- **Advertisement** — przechowuje materiały reklamowe wyświetlane na ekranie klienta w trybie karuzeli.
- **User** — reprezentuje użytkownika systemu (kasjer lub administrator) z poświadczeniami logowania i przypisaną rolą.

Konfiguracja mapowania i współbieżność

Wszystkie encje są precyzyjnie mapowane za pomocą atrybutów adnotacji danych (*Data Annotations*) [22] zdefiniowanych w przestrzeni nazw System.ComponentModel.DataAnnotations.Schema, takich jak [Table], [Key], [ForeignKey], [Required] czy [Timestamp]. Stosowanie adnotacji deklaratywnych — jako alternatywy dla konfiguracji płynnej (*Fluent API*) — zapewnia czytelność definicji encji i samodokumentowanie się modelu domenowego.

Z perspektywy inżynierii systemów wielostanowiskowych kluczowym mechanizmem zastosowanym na poziomie definicji schematu bazy danych jest **optymistyczne blokowanie współbieżności** (*Optimistic Concurrency Control*) [23]. W encjach Reservation i Ticket zdefiniowano właściwość RowVersion opatrzoną atrybutem [Timestamp]:

1Stazy / CA_TEST / Models / Reservation.cs

```
[Timestamp]

public byte[]? RowVersion { get; set; }
```

Wartość RowVersion jest automatycznie inkrementowana przez silnik bazodanowy przy każdej modyfikacji rekordu. Jeśli dwóch kasjerów podejmie próbę jednoczesnej aktualizacji tego samego wiersza (np. rezerwacji tego samego miejsca), Entity Framework Core wykryje rozbieżność wersji i zgłosi wyjątek DbUpdateConcurrencyException dla transakcji, która jako druga podejmie próbę zapisu [23]. Mechanizm ten skutecznie zapobiega anomalii podwójnej sprzedaży (*double-booking*) bez konieczności stosowania pesymistycznych blokad na poziomie bazy danych, które obniżałyby przepustowość systemu POS przy wielu równoległych stanowiskach kasjerskich.

Bibliografia (kontynuacja numeracji)

[1] IEEE, *IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications*, New York: IEEE, 1998.

[2] I. Sommerville, *Software Engineering*, 10th ed., Harlow: Pearson, 2015.

[3] D. Ferraiolo, R. Kuhn, „Role-Based Access Controls," w *Proceedings of 15th National Computer Security Conference*, Baltimore, 1992, s. 554–563.

[4] D. Parmenter, *Key Performance Indicators: Developing, Implementing, and Using Winning KPIs*, 4th ed., Hoboken: Wiley, 2019.

[5] Microsoft, „What is .NET?," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/core/introduction

[6] Microsoft, „The MVVM pattern," .NET Application Architecture Guide, Microsoft Learn, 2023. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm

[7] The PostgreSQL Global Development Group, „PostgreSQL Documentation," 2024. [Online]. Dostępne: https://www.postgresql.org/docs/

[8] SQLite Consortium, „SQLite Documentation," 2024. [Online]. Dostępne: https://www.sqlite.org/docs.html

[9] Microsoft, „Entity Framework Core overview," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/

[10] Microsoft, „WPF overview," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/

[11] Microsoft, „Data binding overview — WPF," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/data/data-binding-overview

[12] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, M. Stal, *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*, Chichester: Wiley, 1996.

[13] Microsoft, „Task-based asynchronous pattern (TAP)," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/standard/asynchronous-programming-patterns/task-based-asynchronous-pattern-tap

[14] Repozytorium projektu — plik App.xaml.cs, linie 62–73 (współdzielony DataContext). [Online]. Dostępne: https://github.com/1Stazy/CA_TEST/blob/main/App.xaml.cs

[15] Microsoft, „MVVM Toolkit introduction," .NET Community Toolkit, Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/

[16] M. Fowler, *Patterns of Enterprise Application Architecture*, Boston: Addison-Wesley Professional, 2002, s. 475–476.

[17] M. Fowler, „Application Controller," w *Patterns of Enterprise Application Architecture*, 2002. [Online]. Dostępne: https://martinfowler.com/eaaCatalog/applicationController.html

[18] Microsoft, „Dependency injection in .NET," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection

[19] M. Seemann, *Dependency Injection in .NET*, 2nd ed., Shelter Island: Manning Publications, 2019, s. 47–52 (*Composition Root*).

[20] Microsoft, „Migrations Overview — EF Core," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/

[21] Npgsql Development Team, „Npgsql — .NET Access to PostgreSQL," 2024. [Online]. Dostępne: https://www.npgsql.org/

[22] Microsoft, „Data Annotations — EF Core," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/modeling/entity-properties

[23] Microsoft, „Handling concurrency conflicts — EF Core," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/saving/concurrency

3.4. Implementacja warstwy danych

Warstwa dostępu do danych (*Data Access Layer*, DAL) została zrealizowana przy wykorzystaniu frameworka **Entity Framework Core** (EF Core) [1] w architekturze **Code-First** [2]. Oznacza to, że struktura relacyjnej bazy danych stanowi bezpośrednie odzwierciedlenie obiektowego modelu domeny (*domain model*) wyrażonego w języku C#. Jak wskazuje dokumentacja Microsoft, podejście Code-First umożliwia programistom definiowanie modelu za pomocą klas języka C#, na podstawie których silnik ORM automatycznie generuje odpowiadający im schemat relacyjny [2]. Takie podejście pozwala na pełne uniezależnienie logiki aplikacji od specyfiki konkretnego dialektu języka SQL, co wpisuje się w zasadę odwrócenia zależności (*Dependency Inversion Principle*) stanowiącą jeden z filarów paradygmatu SOLID opisywanego przez Martina [3].

3.4.1. Modele encji

Sercem warstwy danych są modele encji (*entity classes*), które realizują paradygmat mapowania obiektowo-relacyjnego (*Object-Relational Mapping*, ORM) [4]. W systemie zdefiniowano **11 klas encji** zlokalizowanych w katalogu Models/, z których każda reprezentuje odrębną tabelę w relacyjnej bazie danych. Pełna lista encji obejmuje: Film, Director, Hall, Seat, Screening, Reservation, Ticket, TicketTypeDefinition, PromoCode, Advertisement oraz User [5].

Każda klasa encji została precyzyjnie skonfigurowana przy użyciu **atrybutów adnotacji danych** (*Data Annotations*) [6] z przestrzeni nazw System.ComponentModel.DataAnnotations oraz System.ComponentModel.DataAnnotations.Schema. Mechanizm adnotacji deklaratywnych pozwala silnikowi mapowania EF Core na jednoznaczne określenie schematu tabeli docelowej, kluczy głównych, ograniczeń integralności oraz relacji pomiędzy encjami. Poniżej przedstawiono reprezentatywne przykłady zastosowanych adnotacji w kontekście kluczowych encji projektu:

- Atrybut [Table("Reservations")] — przypisany do klasy Reservation — jawnie określa nazwę tabeli docelowej w bazie danych, eliminując zależność od konwencji nazewniczych frameworka [6].
- Atrybut [Key] — zastosowany do właściwości Id we wszystkich encjach — definiuje jednoznaczny klucz główny (*primary key*) każdej tabeli.
- Atrybut [Required] — nałożony m.in. na właściwości CustomerName i CreatedAt encji Reservation — wymusza na poziomie schematu bazy danych ograniczenie NOT NULL, zapobiegając zapisowi rekordów z niekompletnymi danymi [6].
- Atrybut [ForeignKey("ScreeningId")] — zastosowany w encji Reservation — precyzyjnie definiuje relację wiele-do-jednego z encją Screening, gwarantując spójność referencyjną (*referential integrity*) i zapobiegając powstawaniu osieroconych rekordów (*orphaned records*) [7].

Szczególną rolę w modelach encji pełni właściwość RowVersion, dodana do kluczowych klas transakcyjnych — Reservation oraz Ticket. Właściwość ta, opatrzona atrybutem [Timestamp], jest mapowana w bazie danych jako specjalny **token współbieżności** (*concurrency token*) [8]:

1Stazy / CA_TEST / Models / Reservation.csv2

[Timestamp]

public byte[]? RowVersion { get; set; }

Każda modyfikacja rekordu opatrzonego takim tokenem powoduje automatyczną aktualizację wartości RowVersion przez silnik bazodanowy. Mechanizm ten stanowi fundament **optymistycznego blokowania współbieżności** (*Optimistic Concurrency Control*, OCC) [8], którego szczegółowe działanie w kontekście procesu rezerwacji opisano w podrozdziale 3.3.

3.4.2. Konfiguracja kontekstu bazy danych (CinemaDbContext)

Centralnym punktem komunikacji pomiędzy warstwą aplikacyjną a serwerem bazy danych jest klasa CinemaDbContext (plik Services/CinemaDbContext.cs), dziedzicząca po bazowej klasie DbContext dostarczanej przez framework EF Core [1]. Zgodnie z dokumentacją Microsoft, klasa DbContext stanowi abstrakcję sesji z bazą danych i jest odpowiedzialna za trzy fundamentalne operacje: śledzenie zmian w encjach (*change tracking*), translację zapytań LINQ na natywne instrukcje SQL (*query translation*) oraz zarządzanie transakcjami (*transaction management*) [9].

Wewnątrz klasy CinemaDbContext zdefiniowano **10 publicznych właściwości** typu DbSet<T>, które pełnią rolę typizowanych kolekcji obiektowych reprezentujących poszczególne tabele bazy danych [1]. Każda właściwość DbSet<T> umożliwia wykonywanie operacji CRUD (*Create, Read, Update, Delete*) na odpowiadającej jej encji za pośrednictwem mechanizmu LINQ [10]:

1Stazy / CA_TEST / Services / CinemaDbContext.cs

```
public DbSet<Film> Films { get; set; }

public DbSet<User> Users { get; set; }

public DbSet<Director> Directors { get; set; }

public DbSet<Hall> Halls { get; set; }

public DbSet<Screening> Screenings { get; set; }

public DbSet<Reservation> Reservations { get; set; }

public DbSet<Ticket> Tickets { get; set; }

public DbSet<PromoCode> PromoCodes { get; set; }

public DbSet<TicketTypeDefinition> TicketTypes { get; set; }

public DbSet<Advertisement> Advertisements { get; set; }
```

Konfiguracja połączenia z bazą danych została zrealizowana w nadpisanej metodzie OnConfiguring, która implementuje **strategię konfiguracji środowiskowej** (*environment-based configuration*). System w pierwszej kolejności odczytuje wartość zmiennej środowiskowej DATABASE_URL — co umożliwia wdrożenie na platformach chmurowych (np. Railway, Render) — a w przypadku jej braku stosuje domyślny łańcuch połączeniowy (*connection string*) wskazujący na lokalną instancję PostgreSQL [11]:

1Stazy / CA_TEST / Services / CinemaDbContext.cs

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)

{

   var connectionString = Environment.GetEnvironmentVariable("DATABASE_URL");
```

```
    if (string.IsNullOrEmpty(connectionString))
    {
        connectionString =
"Host=localhost;Database=cinema_db;Username=postgres;Password=postgres;Port=5432";
    }

    optionsBuilder.UseNpgsql(
        connectionString,
        options => options
            .EnableRetryOnFailure(3, TimeSpan.FromSeconds(10), new[] { "42P01" })
            .CommandTimeout(30)
    );

    #if DEBUG
    optionsBuilder.EnableSensitiveDataLogging(true);
    #endif
}
```

Z powyższego listingu wynika kilka istotnych decyzji architektonicznych. Po pierwsze, dostawca bazy danych Npgsql [11] został skonfigurowany z mechanizmem **automatycznego ponawiania transakcji** (*execution strategy with retry*) — w przypadku wystąpienia przejściowego błędu połączenia system podejmie do 3 prób ponownego wykonania operacji z 10-sekundowym interwałem [1]. Po drugie, limit czasu wykonania polecenia (*command timeout*) ustalono na 30 sekund, co stanowi kompromis pomiędzy tolerancją na złożone zapytania raportowe a szybkością wykrywania awarii połączenia. Po trzecie, w trybie kompilacji debugowej (#if DEBUG) aktywowane jest logowanie wrażliwych danych zapytań (EnableSensitiveDataLogging), co ułatwia diagnostykę podczas fazy deweloperskiej, lecz jest automatycznie wyłączane w kompilacji produkcyjnej ze względów bezpieczeństwa [1].

Warto odnotować, że mimo iż bieżąca konfiguracja metody OnConfiguring wykorzystuje wyłącznie dostawcę Npgsql (PostgreSQL), plik migracji Migrations/20260106155819_InitialCreate.cs obecny w repozytorium zawiera typy kolumn charakterystyczne dla SQLite (INTEGER, TEXT, REAL), co potwierdza, że schemat bazy danych był pierwotnie generowany z wykorzystaniem dostawcy SQLite — co jest zgodne z deklarowaną w założeniach projektowych elastycznością środowiska uruchomieniowego [5].

Wszelkie modyfikacje struktury modeli domenowych są propagowane do fizycznego schematu bazy danych za pomocą **mechanizmu migracji** EF Core [12]. Narzędzie wiersza poleceń dotnet ef migrations add <NazwaMigracji> generuje pliki migracji zawierające instrukcje różnicowe (*differential schema changes*), które następnie są aplikowane poleceniem dotnet ef database update. Podejście to zapewnia pełną wersjonowalność schematu bazy danych w systemie kontroli wersji, umożliwiając deterministyczne odtworzenie dowolnego stanu bazy w przyszłości [12].

3.5. Implementacja logiki aplikacji

Logika aplikacji została oddzielona od warstwy interfejsu użytkownika przy wykorzystaniu wzorca architektonicznego **Model–View–ViewModel** (MVVM) [13]. Przepływ danych i sterowanie architekturą opiera się na konsekwentnym rozdziale odpowiedzialności (*Separation of Concerns*) [14] na klasy modeli widoku (*ViewModels*) oraz autonomiczne serwisy usługowe (*Services*).

3.5.1. ViewModele i logika biznesowa

Podstawą implementacji warstwy logiki prezentacji jest biblioteka **CommunityToolkit.Mvvm** [15], rozwijana przez Microsoft w ramach ekosystemu .NET Community Toolkit. Wszystkie klasy obsługujące poszczególne ekrany aplikacji dziedziczą po wspólnej klasie bazowej ViewModelBase (plik Core/ViewModelBase.cs), która z kolei rozszerza klasę ObservableObject dostarczaną przez ww. bibliotekę:

1Stazy / CA_TEST / Core / ViewModelBase.cs

```
public class ViewModelBase : ObservableObject

{

}
```

Zastosowanie tego wzorca odpowiada koncepcji *Layer Supertype* opisanej przez Fowlera [16] — wspólna klasa bazowa stanowi centralne miejsce do ewentualnego wprowadzenia funkcjonalności przekrojowych (*cross-cutting concerns*), takich jak logowanie diagnostyczne czy wspólne mechanizmy walidacji, bez konieczności modyfikacji poszczególnych ViewModeli.

W repozytorium zidentyfikowano **12 klas ViewModeli** [5], których nazewnictwo jest ściśle skorelowane z odpowiadającymi im widokami XAML:

ViewModel	Odpowiedzialność
MainViewModel	Orkiestrator aplikacji — zarządza nawigacją i synchronizacją okien kasjera/klienta
LoginViewModel	Obsługa procesu uwierzytelniania
DashboardViewModel	Panel nawigacyjny kasjera z kontrolą routingu widoków
MoviesViewModel	Katalog filmów z filtrowaniem, wyszukiwaniem i karuzelą
MovieDetailViewModel	Szczegóły filmu i lista dostępnych seansów
SeatSelectionViewModel	Interaktywna mapa miejsc z selekcją
TicketSummaryViewModel	Podsumowanie koszyka, kody promocyjne, finalizacja transakcji
TicketManagementViewModel	Zarządzanie biletami — zwroty, edycja danych, zmiana miejsc
ScheduleViewModel	Harmonogram seansów

ViewModel	Odpowiedzialność
ReportsViewModel	Moduł analityczny z wizualizacją KPI
ClientScheduleViewModel	Widok harmonogramu na ekranie klienta
GenreItem	Klasa pomocnicza — element filtra gatunków

Kluczową innowacją zastosowaną w kodzie jest wykorzystanie **generatorów kodu źródłowego** (*Source Generators*) [17], mechanizmu wprowadzonego w C# 9.0 i platformie .NET 5, umożliwiającego generowanie dodatkowego kodu w trakcie kompilacji na podstawie analizy drzewa składniowego Roslyn. Zamiast ręcznej i podatnej na błędy implementacji interfejsu INotifyPropertyChanged dla każdego pola publicznego — co w literaturze określa się mianem *boilerplate code* [15] — zastosowano atrybut [ObservableProperty]. Podczas kompilacji generator CommunityToolkit automatycznie tworzy publiczną właściwość z akcesorami i mutatorami, które powiadamiają widok o zmianie stanu za pośrednictwem zdarzenia PropertyChanged. Na przykładzie klasy MainViewModel:

1Stazy / CA_TEST / ViewModels / MainViewModel.cs

```
[ObservableProperty]

private object _currentView;


[ObservableProperty]

private object _clientView;
```

Powyższa deklaracja dwóch prywatnych pól z atrybutem [ObservableProperty] powoduje wygenerowanie w trakcie kompilacji publicznych właściwości CurrentView i ClientView z pełną implementacją mechanizmu notyfikacji — co redukuje objętość kodu klasy o szacunkowo 60–70% w porównaniu z implementacją ręczną [15].

Analogicznie rozwiązano kwestię obsługi komend użytkownika (*user commands*). Zastosowanie atrybutu [RelayCommand] nad metodami — w tym metodami asynchronicznymi typu async Task — powoduje automatyczne wygenerowanie implementacji interfejsu ICommand [18]. Wygenerowana komenda wspiera zarówno operacje synchroniczne, jak i asynchroniczne, a w przypadku wariantu asynchronicznego automatycznie zarządza flagą IsBusy, co minimalizuje ryzyko blokowania głównego wątku dispatera WPF (*UI thread*) [13]. Przykład z klasy TicketSummaryViewModel:

1Stazy / CA_TEST / ViewModels / TicketSummaryViewModel.cs

```
[RelayCommand]

private async Task FinalizeTransaction()
```

Logika biznesowa ViewModeli obejmuje w szczególności: obliczanie sumarycznej kwoty koszyka z uwzględnieniem dynamicznej konfiguracji typów biletów i kodów promocyjnych (klasa TicketSummaryViewModel), filtrowanie i wyszukiwanie w katalogu filmów z obsługą wielu kryteriów

jednocześnie (klasa MoviesViewModel), koordynację procesu zwrotów z walidacją temporalną — system blokuje zwroty na mniej niż 10 minut przed seansem (klasa TicketManagementViewModel) — oraz zarządzanie nawigacją pomiędzy ekranami za pośrednictwem wzorca *Application Controller* [19] zaimplementowanego w klasie MainViewModel.

3.5.2. Usługi systemowe (*Services*)

Aby uchronić klasy ViewModeli przed nadmiernym rozrostem odpowiedzialności — naruszeniem zasady pojedynczej odpowiedzialności (*Single Responsibility Principle*, SRP) [3] — złożone operacje infrastrukturalne i biznesowe oddelegowano do dedykowanych **serwisów usługowych**. Zgodnie z analizą repozytorium, warstwa usług jest zlokalizowana w katalogu Services/ i obejmuje następujące komponenty [5]:

Usługa uwierzytelniania (AuthenticationService) — realizuje proces weryfikacji tożsamości użytkowników na podstawie poświadczeń przechowywanych w tabeli Users. W bieżącej implementacji klasa udostępnia metodę ValidateUser(string username, string password), która odpytuje bazę danych za pośrednictwem kontekstu CinemaDbContext i zwraca obiekt User w przypadku pozytywnej walidacji [5]. Na podstawie właściwości Role zwróconego obiektu klasa DashboardViewModel realizuje kontrolę dostępu opartą na rolach (*Role-Based Access Control*) [20], różnicując interfejs kasjera od interfejsu administratora.

Usługi generowania dokumentów PDF (TicketPdfGenerator, PdfReportService) — hermetyzują logikę biblioteki QuestPDF [21] w dedykowanych klasach zlokalizowanych w podkatalogu Services/Pdf/. Klasa TicketPdfGenerator odpowiada za deklaratywną kompozycję spersonalizowanych biletów kinowych zawierających kody QR, natomiast PdfReportService generuje wielostronicowe raporty analityczne z wizualizacjami KPI. Szczegółowy opis tych komponentów przedstawiono w podrozdziale 2.7.1.

Usługa poczty elektronicznej (EmailService) — hermetyzuje logikę komunikacji z zewnętrznym serwerem SMTP za pośrednictwem klas przestrzeni nazw System.Net.Mail [22]. Serwis odbiera żądanie wysyłki, konstruuje obiekt MailMessage z załącznikiem w postaci pliku PDF biletu, a następnie realizuje asynchroniczną transmisję na skonfigurowany serwer pocztowy (w bieżącej implementacji smtp.gmail.com:587 z szyfrowaniem TLS). Asynchroniczność operacji zapewnia, że interfejs kasjera nie jest blokowany w trakcie oczekiwania na odpowiedź serwera SMTP.

Menedżer ekranów (ScreenManager) — odpowiada za integrację z natywnym interfejsem API systemu Windows w zakresie obsługi konfiguracji wielomonitorowej. Klasa wykorzystuje typ Screen.AllScreens z przestrzeni nazw System.Windows.Forms [23] do programistycznego wykrycia wszystkich podłączonych urządzeń wyświetlających, a następnie identyfikuje monitor niebędący wyświetlaczem głównym (!s.Primary) i automatycznie przenosi na niego okno CustomerWindow. Metoda OpenCustomerWindow realizuje precyzyjne pozycjonowanie okna poprzez ustawienie współrzędnych (Left, Top) na wartości odpowiadające obszarowi roboczemu (*WorkingArea*) docelowego ekranu, wymuszenie rozmiaru okna na pełny wymiar monitora oraz maksymalizację z wyłączeniem belki systemowej (WindowStyle.None) [5]:

/ Services / ScreenManager.cs:

```
var screens = Screen.AllScreens;

var targetScreen = screens.FirstOrDefault(s => !s.Primary) ?? screens.FirstOrDefault();
```

```
if (targetScreen != null)
{
    customerWindow.WindowState = WindowState.Normal;
    customerWindow.WindowStyle = WindowStyle.None;

    customerWindow.Left = targetScreen.WorkingArea.Left;
    customerWindow.Top = targetScreen.WorkingArea.Top;

    customerWindow.Width = targetScreen.Bounds.Width;
    customerWindow.Height = targetScreen.Bounds.Height;

    customerWindow.Show();
    customerWindow.WindowState = WindowState.Maximized;
}
```

Powyższa implementacja zawiera mechanizm *fallback* — w przypadku braku drugiego monitora (np. podczas pracy deweloperskiej) okno klienta jest wyświetlane na monitorze głównym w trybie normalnym, co zapewnia funkcjonalność aplikacji niezależnie od konfiguracji sprzętowej stanowiska.

Usługa konfiguracji raportów (ReportSettingsService) — zarządza parametrami modułu raportowego, udostępniając spójny interfejs dostępu do ustawień generowania dokumentów analitycznych.

Usługi inicjalizacji danych (DatabaseGenerator, DataSeeder) — odpowiadają za automatyczne tworzenie schematu bazy danych przy pierwszym uruchomieniu aplikacji oraz wypełnianie go danymi testowymi i historycznymi, co umożliwia natychmiastową demonstrację funkcjonalności modułu raportowego bez konieczności ręcznego wprowadzania danych.

Konwertery wartości (*Value Converters*) — trzy dedykowane klasy w podkatalogu Services/Converters/ (BooleanToVisibilityConverter, BooleanToColorConverter, IsFutureDateConverter) implementują interfejs IValueConverter [24] i realizują transformację danych pomiędzy warstwą ViewModelu a warstwą widoku XAML. Konwertery te są rejestrowane jako globalne zasoby aplikacji w pliku App.xaml, co umożliwia ich wielokrotne wykorzystanie w różnych widokach bez duplikacji kodu.

Bibliografia (kontynuacja numeracji)

[1] Microsoft, „Entity Framework Core overview," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/

[2] Microsoft, „Creating and configuring a model — Code First," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/modeling/

[3] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Upper Saddle River: Prentice Hall, 2017.

[4] C. J. Date, *An Introduction to Database Systems*, 8th ed., Boston: Pearson, 2003.

[5] Repozytorium projektu — 1Stazy/CA_TEST. [Online]. Dostępne: https://github.com/1Stazy/CA_TEST

[6] Microsoft, „Data Annotations — EF Core," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/modeling/entity-properties

[7] Microsoft, „Relationships — EF Core," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/modeling/relationships

[8] Microsoft, „Handling concurrency conflicts — EF Core," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/saving/concurrency

[9] Microsoft, „DbContext Lifetime, Configuration, and Initialization," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/dbcontext-configuration/

[10] Microsoft, „Language Integrated Query (LINQ)," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/csharp/linq/

[11] Npgsql Development Team, „Npgsql — .NET Access to PostgreSQL," 2024. [Online]. Dostępne: https://www.npgsql.org/

[12] Microsoft, „Migrations Overview — EF Core," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/

[13] Microsoft, „The MVVM pattern," Microsoft Learn, 2023. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm

[14] I. Sommerville, *Software Engineering*, 10th ed., Harlow: Pearson, 2015.

[15] Microsoft, „MVVM Toolkit introduction," .NET Community Toolkit, Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/

[16] M. Fowler, *Patterns of Enterprise Application Architecture*, Boston: Addison-Wesley Professional, 2002.

[17] Microsoft, „Source Generators overview," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/source-generators-overview

[18] Microsoft, „RelayCommand attribute," .NET Community Toolkit, Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/generators/relaycommand

[19] M. Fowler, „Application Controller," *Patterns of Enterprise Application Architecture*, 2002. [Online]. Dostępne: https://martinfowler.com/eaaCatalog/applicationController.html

[20] D. Ferraiolo, R. Kuhn, „Role-Based Access Controls," w *Proceedings of 15th National Computer Security Conference*, Baltimore, 1992, s. 554–563.

[21] QuestPDF Contributors, „QuestPDF — Open Source .NET Library for PDF Document Generation," 2024. [Online]. Dostępne: https://www.questpdf.com/

[22] Microsoft, „SmtpClient Class," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/api/system.net.mail.smtpclient

[23] Microsoft, „Screen Class," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/api/system.windows.forms.screen

[24] Microsoft, „IValueConverter Interface," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/api/system.windows.data.ivalueconverter

---

3.6. Implementacja warstwy prezentacji

Warstwa prezentacji (*Presentation Layer*) została zbudowana w oparciu o silnik **Windows Presentation Foundation** (WPF) [1], co pozwoliło na wykorzystanie wektorowej, sprzętowo akcelerowanej grafiki opartej na bibliotece DirectX [2] oraz pełną separację deklaratywnego opisu interfejsu od logiki biznesowej — zgodnie z założeniami wzorca architektonicznego MVVM opisanymi w podrozdziale 2.3.

3.6.1. Interfejs użytkownika w technologii WPF

Główny interfejs aplikacji został zdefiniowany deklaratywnie przy użyciu języka znaczników **XAML** (*Extensible Application Markup Language*) [3]. Całość systemu wizualnego opiera się na centralnym pliku App.xaml, w którym zdefiniowano globalne słowniki zasobów (*Resource Dictionaries*) [4]. Zawarto tam zunifikowane style dla kontrolek (m.in. przycisków, pól tekstowych, nagłówków), definicje palety kolorów dla trybu ciemnego (*dark theme*) stosowanego w module raportowym, a także konfiguracje globalnych konwerterów wartości, które są rejestrowane jako zasoby dostępne dla wszystkich widoków aplikacji.

Architektura okienkowa systemu opiera się na **dwóch głównych kontenerach**: MainWindow.xaml — przeznaczonym do obsługi panelu kasjera i administratora — oraz CustomerWindow.xaml — służącym jako pasywny widok podglądu skierowany do klienta stojącego przy okienku kasowym. Oba okna współdzielą tę samą instancję klasy MainViewModel jako kontekst danych (*DataContext*), co zapewnia automatyczną synchronizację stanu pomiędzy stanowiskiem operatora a ekranem ekspozycji klienta [5].

Wewnątrz MainWindow zaimplementowano mechanizm nawigacji oparty na **wstrzykiwaniu odpowiednich widoków** do centralnego kontenera typu ContentControl [1]. Właściwość Content tego kontenera jest powiązana (*binding*) z właściwością CurrentView klasy MainViewModel, dzięki czemu przypisanie nowej instancji ViewModelu do tej właściwości automatycznie powoduje wyrenderowanie odpowiadającego mu widoku za pośrednictwem mechanizmu niejawnych szablonów danych (*implicit DataTemplates*) [4]. Podejście to umożliwia płynne przełączanie się między modułami aplikacji — np. LoginView, DashboardView, MoviesView, SeatSelectionView, TicketSummaryView, ReportsView — bez konieczności otwierania nowych okien systemowych i bez narastania stosu okien w pamięci operacyjnej.

3.6.2. Konwertery i mechanizmy powiązań danych

Wyeliminowanie bezpośredniego odwoływania się do kontrolek w kodzie strukturalnym (*code-behind*) osiągnięto dzięki zaawansowanemu systemowi **wiązania danych** (*Data Binding*) [6]. Właściwości kontrolek w plikach XAML — takie jak Text, Visibility, Background, ItemsSource — są deklaratywnie powiązane ze stanem odpowiednich właściwości w klasach ViewModel za pośrednictwem wyrażeń {Binding}. Mechanizm ten — oparty na wzorcu obserwatora (*Observer*

Pattern) [7] — automatycznie propaguje zmiany wartości w obu kierunkach (tryby OneWay, TwoWay, OneWayToSource), eliminując konieczność ręcznej manipulacji drzewem wizualnym (*visual tree*) z poziomu kodu C#.

Ponieważ dane przechowywane w klasach ViewModel często nie odpowiadają bezpośrednio typom wizualnym wymaganym przez kontrolki XAML, w projekcie zaimplementowano **niestandardowe konwertery** (*custom Value Converters*) implementujące interfejs IValueConverter [8]. Konwertery te zlokalizowano w podkatalogu Services/Converters/ i zarejestrowano jako globalne zasoby w pliku App.xaml, co umożliwia ich wielokrotne wykorzystanie w dowolnym widoku aplikacji.

Kluczowym konwerterem jest BooleanToColorConverter, który odbiera informację o statusie miejsca na sali kinowej (typ wyliczeniowy SeatStatus) i w locie przekształca ją na odpowiedni obiekt pędzla SolidColorBrush — renderując miejsce na interaktywnej mapie sali w kolorze zielonym (wolne), czerwonym (zarezerwowane) lub niebieskim (wybrane przez kasjera). Analogicznie wykorzystano konwerter BooleanToVisibilityConverter do dynamicznego ukrywania i wyświetlania elementów interfejsu — np. wskaźnika postępu ładowania danych (*ProgressBar*), panelu edycji w module zarządzania biletami, czy panelu rabatów — na podstawie stanu właściwości logicznych IsBusy i IsEditing w odpowiednich ViewModelach. Trzeci konwerter — IsFutureDateConverter — służy do wizualnego różnicowania seansów przyszłych od przeszłych w widoku harmonogramu.

3.7. Realizacja procesu rezerwacji biletów

Główny przepływ biznesowy systemu — proces sprzedaży i rezerwacji biletów — stanowi punkt integracji wszystkich warstw aplikacji i realizuje kompletny scenariusz użycia (*use case*) klasy *critical user flow* [9]. Proces ten inicjowany jest w widoku katalogu filmów (MoviesView), gdzie kasjer przegląda i filtruje dostępne produkcje, a następnie przechodzi do szczegółów wybranego filmu (MovieDetailView) i wybiera konkretny seans z listy nadchodzących projekcji.

Interaktywna mapa miejsc

Logika wyboru miejsc jest podzielona pomiędzy widok SeatSelectionView (plik Views/SeatSelectionView.xaml.cs) a odpowiadający mu ViewModel SeatSelectionViewModel (plik ViewModels/SeatSelectionViewModel.cs). Widok obsługuje zaawansowane mechanizmy interakcji z użytkownikiem, stanowiące uzasadniony wyjątek od reguły minimalizacji kodu *code-behind* — ponieważ operacje związane z przechwytywaniem zdarzeń myszy i rysowaniem prostokąta zaznaczenia na płótnie (*Canvas*) wymagają bezpośredniego dostępu do drzewa wizualnego WPF [1].

System oferuje dwa tryby selekcji miejsc:

1. **Kliknięcie pojedyncze** — realizowane za pośrednictwem komendy ToggleSeatCommand powiązanej z przyciskiem każdego miejsca na mapie, przełączającej status miejsca pomiędzy Available a Selected.
2. **Zaznaczenie prostokątne** (*drag-to-select*) — zaimplementowane w kodzie *code-behind* widoku za pomocą obsługi zdarzeń OnMouseDown, OnMouseMove i OnMouseUp. Po rozpoczęciu przeciągania system zapisuje stan zaznaczenia wszystkich miejsc (_initialSelectionState), dynamicznie rysuje prostokąt zaznaczenia (SelectionBox — element Rectangle na Canvas) i w trakcie ruchu myszy aktualizuje status miejsc na podstawie testu przecięcia (*hit-testing*) ich pozycji wizualnej z obszarem prostokąta:

1Stazy / CA_TEST / Views / SeatSelectionView.xaml.cs

```
private void OnMouseMove(object sender, System.Windows.Input.MouseEventArgs e)
{
    if (!_isDragging) return;

    var currentPoint = e.GetPosition(SelectionArea);
```

Kalkulacja koszyka i kody promocyjne

Po wybraniu odpowiednich miejsc kasjer przechodzi do ekranu podsumowania (TicketSummaryView), gdzie klasa TicketSummaryViewModel inicjuje proces kalkulacji koszyka. Silnik reguł biznesowych realizuje następujące operacje:

- **Przypisanie typów biletów** — każde miejsce w koszyku (TicketCartItem) posiada własny wybór typu biletu (TicketTypeDefinition) z dynamicznie pobieranej z bazy danych listy aktywnych typów. Cena biletu jest obliczana jako iloczyn ceny bazowej seansu i mnożnika typu (PriceMultiplier).
- **Aplikacja kodów rabatowych** — metoda ApplyPromoCode waliduje wprowadzony kod promocyjny z tabelą PromoCodes w bazie danych, obsługując zarówno rabaty procentowe (IsPercentage = true), jak i kwotowe. W przypadku rabatu kwotowego jego wartość jest sekwencyjnie dystrybuowana pomiędzy kolejne pozycje koszyka.
- **Automatyczne generowanie voucherów** — za każde 2 zakupione bilety system automatycznie dołącza do dokumentu PDF voucher na darmowy zestaw barowy (*popcorn + napój*), realizując strategię *loyalty incentive* na poziomie logiki biletowej.

Finalizacja transakcji i kontrola współbieżności

Finalizacja transakcji realizowana jest w metodzie CreateReservationWithTransactionAsync w ramach **jawnej transakcji bazodanowej** (BeginTransactionAsync), co zapewnia atomowość operacji zgodną z regułami ACID [10]. Metoda ta sekwencyjnie: (1) tworzy rekord rezerwacji (Reservation), (2) zapisuje go w celu uzyskania wygenerowanego klucza głównego, (3) iteruje po elementach koszyka, tworząc rekordy biletów (Ticket) z przeliczonymi cenami po uwzględnieniu rabatów, i (4) zatwierdza transakcję (CommitAsync).

To w tym momencie ujawnia się praktyczne zastosowanie mechanizmu **śledzenia tokenów RowVersion** opisanego w podrozdziale 3.4.1. Jeśli pomiędzy momentem wyświetlenia mapy sali a kliknięciem przycisku „Potwierdź rezerwację" inne stanowisko kasowe zdążyło zarezerwować wybrane miejsce, silnik Entity Framework Core wykrywa rozbieżność wersji rekordu i zgłasza wyjątek DbUpdateConcurrencyException [11]. Aplikacja przechwytuje ten wyjątek, wycofuje transakcję (RollbackAsync) i informuje kasjera o niedostępności miejsc za pośrednictwem dedykowanego okna dialogowego:

```
1Stazy / CA_TEST / ViewModels / TicketSummaryViewModel.csv2

catch (Microsoft.EntityFrameworkCore.DbUpdateConcurrencyException)
{
    await transaction.RollbackAsync();
```

```
    System.Windows.MessageBox.Show("Konflikt dostępu: inny użytkownik zmienił dane. Spróbuj ponownie.", "Błąd współbieżności");
    return null;
}
```

Analogiczny mechanizm obsługi konfliktu współbieżności zaimplementowano w module zarządzania biletami (TicketManagementViewModel) — zarówno w operacji edycji danych klienta (SaveEdit), jak i w operacjach zwrotu pojedynczego biletu (RefundSingleTicket) oraz anulowania całej rezerwacji (RefundFullReservation). W przypadku zwrotów system realizuje dodatkowo **walidację temporalną** — operacja jest blokowana, jeśli do rozpoczęcia seansu pozostało mniej niż 10 minut, co zapobiega nadużyciom proceduralnym.

3.8. Implementacja dodatkowych funkcjonalności systemu

W celu podniesienia ergonomii pracy kasjerów oraz rozszerzenia możliwości obsługi posprzedażowej klienta, standardową architekturę systemu wzbogacono o dodatkowe moduły peryferyjne i interfejsowe.

3.8.1. Obsługa wielu monitorów

System został zaprojektowany do pracy w środowisku dwumonitorowym (*dual-display*), charakterystycznym dla stanowisk typu POS w branży usługowej [12]. Mechanizm zarządzania ekranami jest zaimplementowany w klasie ScreenManager (plik Services/ScreenManager.cs) z wykorzystaniem natywnych klas przestrzeni nazw System.Windows.Forms [13]. Logika usługi podczas inicjalizacji aplikacji — wywoływanej z metody OnStartup w pliku App.xaml.cs — iteruje po kolekcji Screen.AllScreens, identyfikując podłączone wyświetlacze fizyczne. W przypadku wykrycia drugiego monitora usługa automatycznie instancjonuje obiekt CustomerWindow, pozycjonuje go na dodatkowym ekranie (ustawiając współrzędne Left i Top na wartości odpowiadające obszarowi roboczemu WorkingArea docelowego monitora), wymusza rozmiar na pełny wymiar ekranu, aktywuje tryb pełnoekranowy bez obramowania (WindowStyle = WindowStyle.None) i maksymalizuje okno [5].

Widok klienta współdzieli kontekst danych z aktywnym ViewModelem kasjera — klasa MainViewModel synchronizuje właściwość ClientView z właściwością CurrentContent klasy DashboardViewModel za pośrednictwem obsługi zdarzenia PropertyChanged. Dzięki temu klient w czasie rzeczywistym widzi na swoim ekranie interaktywny podgląd wybieranych miejsc, szczegóły wybranego filmu, podsumowanie zamówienia z kwotą do zapłaty, a w stanach bezczynności — automatycznie rotowane materiały reklamowe pobrane z tabeli Advertisements oraz harmonogram seansów (wyświetlany przez 30 sekund po aktywacji przez kasjera, z użyciem obiektu DispatcherTimer).

3.8.2. Generowanie biletów w formacie PDF

Po pomyślnej autoryzacji transakcji system generuje elektroniczne bilety za pośrednictwem klasy TicketPdfGenerator (plik Services/Pdf/TicketPdfGenerator.cs), wykorzystującej bibliotekę **QuestPDF** [14] w wersji Community. Architektura QuestPDF opiera się na deklaratywnym **interfejsie płynnym** (*Fluent API*), który umożliwia projektowanie układu dokumentu bezpośrednio w kodzie C# z zachowaniem pełnej kontroli typów w czasie kompilacji [14].

Układ graficzny biletu, kompilowany w metodzie GenerateTicketPdf, obejmuje trzy sekcje dokumentu formatu A4:

- **Nagłówek** — dwukolumnowy układ (*Row*) zawierający nazwę systemu „KINO SYSTEM" z numerem transakcji oraz obraz kodu QR o szerokości 85 pikseli, wygenerowany przez wewnętrzną metodę GenerateQrCode.
- **Zawartość** — sekcja szczegółów seansu (tytuł filmu, data, godzina, sala) otoczona ramką z tłem Colors.Grey.Lighten5, a poniżej tabelaryczne zestawienie zarezerwowanych miejsc z cenami indywidualnymi i sumą transakcji.
- **Stopka** — informacje adresowe kina oraz instrukcja dla klienta dotycząca okazania kodu QR przy wejściu na salę.

Dodatkowo, za każde 2 zakupione bilety system automatycznie dołącza sekcję **vouchera promocyjnego** — graficznie wyróżniony blok „GOLDEN TICKET" z odrębnym kodem QR (prefiks VOUCHER-), uprawniający do odbioru darmowego zestawu barowego. Mechanizm ten zaimplementowano w metodzie ComposeVoucherPage, a warunek jego aktywacji (voucherAmount = tickets.Count / 2) realizuje prostą regułę lojalnościową na poziomie generatora dokumentów.

3.8.3. Integracja kodów QR

W strukturę każdego generowanego biletu PDF wstrzykiwany jest unikalny kod QR (*Quick Response Code*) [15]. Generowanie realizowane jest przez metodę prywatną GenerateQrCode klasy TicketPdfGenerator, która wykorzystuje zarządzaną bibliotekę **QRCoder** [16]:

1Stazy / CA_TEST / Services / Pdf / TicketPdfGenerator.cs

```
private byte[] GenerateQrCode(string text)

{

   if (string.IsNullOrEmpty(text)) text = "000000";



   using (QRCodeGenerator qrGenerator = new QRCodeGenerator())

   using (QRCodeData qrCodeData = qrGenerator.CreateQrCode(text,
QRCodeGenerator.ECCLevel.Q))
```

Metoda poddaje transformacji unikalny numer transakcji (TransactionNumber — skrócony identyfikator GUID przypisany do rezerwacji) na tablicę bajtów (byte[]) reprezentującą obraz PNG kodu QR. Zastosowano poziom korekcji błędów ECCLevel.Q (poziom kwartylowy — odporność na uszkodzenie do 25% powierzchni kodu) [15], co zapewnia niezawodną detekcję nawet przy częściowym zabrudzeniu lub niedoskonałym wydruku. Klasa PngByteQRCode generuje mapę bitową bezpośrednio w pamięci operacyjnej, a wynikowa tablica bajtów jest natychmiast osadzana w dokumencie QuestPDF za pośrednictwem metody .Image(qrBytes). Podejście to eliminuje konieczność zapisywania plików graficznych na dysku twardym, oszczędzając zasoby wejścia/wyjścia (*I/O*) stanowiska kasowego i przyspieszając proces finalizacji transakcji.

3.8.4. System powiadomień e-mail

Mechanizm dystrybucji biletów elektronicznych do klientów jest realizowany za pośrednictwem klasy EmailService (plik Services/EmailService.cs), która hermetyzuje komunikację z zewnętrznym

serwerem SMTP [17] z wykorzystaniem natywnych klas platformy .NET z przestrzeni nazw System.Net.Mail [18].

Serwis udostępnia metodę SendTicketEmail(string recipientEmail, string userName, string pdfPath), która realizuje następujący przepływ:

1. Konstrukcja obiektu MailMessage z nadawcą „Kino System", tematem „Twój bilet do Kina System" oraz spersonalizowaną treścią powitania.
2. Warunkowe dołączenie wygenerowanego pliku PDF jako obiektu Attachment — z weryfikacją istnienia pliku na dysku (File.Exists(pdfPath)).
3. Konfiguracja klienta SMTP (SmtpClient) z parametrami serwera Gmail (smtp.gmail.com, port 587), uwierzytelnieniem za pośrednictwem obiektu NetworkCredential oraz włączonym szyfrowaniem TLS (EnableSsl = true) [18].
4. Wysyłka wiadomości i zwolnienie zasobów (jawne Dispose() obiektu Attachment zapobiegające blokadom pliku).

Cały proces może być realizowany asynchronicznie — wywołanie SendTicketEmail z poziomu ViewModelu odbywa się w wątku tła (Task.Run), co gwarantuje, że interfejs kasjera nie jest blokowany w trakcie oczekiwania na odpowiedź serwera SMTP. Mechanizm ten jest istotny z perspektywy ergonomii stanowiska kasowego, gdzie czas odpowiedzi interfejsu (*UI responsiveness*) jest krytycznym parametrem jakościowym [19].

3.8.5. Dodatkowe komponenty interfejsu użytkownika

Warstwa wizualna została wzbogacona o niestandardowe mechanizmy renderujące, mające na celu uatrakcyjnienie doświadczenia użytkownika (*User Experience*, UX). Najbardziej złożonym komponentem jest **moduł karuzeli 3D** (*CoverFlow effect*) zaimplementowany jako reużywalny komponent UserControl w pliku Views/Controls/Carousel3DView.xaml.cs.

Kontrolka ta wykorzystuje podsystem grafiki trójwymiarowej WPF oparty na klasach z przestrzeni nazw System.Windows.Media.Media3D [20]. Architektura renderowania obejmuje:

- **Viewport3D** (MainViewport) — kontener sceny trójwymiarowej osadzony w drzewie wizualnym XAML.
- **PerspectiveCamera** (Camera) — wirtualna kamera z konfigurowalną odległością od sceny (CameraDistance — zdefiniowana jako DependencyProperty [1] z wartością domyślną 12.0), symulująca perspektywę obserwatora.
- **MeshGeometry3D** — prostokątna siatka geometryczna (*mesh*) o 4 wierzchołkach i 2 trójkątach, tworząca płaską powierzchnię, na którą nakładana jest tekstura plakatu filmowego.
- **DiffuseMaterial** — materiał z nałożoną teksturą bitmapy plakatu (wczytaną z właściwości PosterUri encji Film) lub wygenerowanym materiałem zastępczym (*debug material*) w przypadku braku plakatu.
- **Transform3DGroup** — kompozycja transformacji obejmująca RotateTransform3D (obrót wokół osi Y o konfigurowalny kąt 60°) i TranslateTransform3D (przesunięcie boczne i w głąb sceny o konfigurowalną odległość).

Metoda UpdatePositions(bool animate) dynamicznie oblicza pozycje docelowe wszystkich elementów karuzeli — wybrany film jest umieszczany centralnie (pozycja X = 0, Z = 0, kąt obrotu = 0°), natomiast elementy boczne są odsuwane na odległość centerGap = 3.5 jednostek, cofane w głąb sceny o zDepth = 3.0 i obracane o ±60°. Przejścia między stanami są realizowane za pomocą animacji DoubleAnimation z funkcją wygładzającą CubicEase i czasem trwania 0.5 sekundy, co zapewnia płynny, sprzętowo akcelerowany efekt rotacji [2]:

1Stazy / CA_TEST / Views / Controls / Carousel3DView.xaml.cs

```
var duration = TimeSpan.FromSeconds(0.5);

var easing = new CubicEase { EasingMode = EasingMode.EaseOut };


var animX = new DoubleAnimation(targetX, duration) { EasingFunction = easing };

var animZ = new DoubleAnimation(targetZ, duration) { EasingFunction = easing };

var animAngle = new DoubleAnimation(targetAngle, duration) { EasingFunction = easing };
```

Interakcja użytkownika z karuzelą jest obsługiwana za pomocą mechanizmu **testowania trafień** (*hit-testing*) [21] — metoda OnViewportMouseDown przekazuje współrzędne kliknięcia do VisualTreeHelper.HitTest, który realizuje rzutowanie promienia (*ray casting*) w scenę 3D. W metodzie zwrotnej ResultCallback system identyfikuje trafiony model geometryczny (RayMeshGeometry3DHitTestResult), odnajduje odpowiadający mu obiekt Film w słowniku _filmModels i aktualizuje właściwość SelectedItem, wyzwalając animację przejścia i nawigację do szczegółów filmu.

Kontrolka Carousel3DView jest stosowana w dwóch kontekstach wizualnych: na ekranie kasjera (MoviesView) — prezentuje polecane filmy z oceną IMDb ≥ 7.0 — oraz na ekranie klienta (CustomerWindow) — wyświetla pełen katalog aktywnych produkcji z automatyczną rotacją sterowaną przez DispatcherTimer w klasie MainViewModel.

Bibliografia (kontynuacja numeracji)

[1] Microsoft, „WPF overview," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/

[2] Microsoft, „Graphics rendering overview — WPF," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/graphics-multimedia/wpf-graphics-rendering-overview

[3] Microsoft, „XAML overview — WPF," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/xaml/

[4] Microsoft, „Resources overview — WPF," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/systems/xaml-resources-overview

[5] Repozytorium projektu — 1Stazy/CA_TEST. [Online]. Dostępne: https://github.com/1Stazy/CA_TEST

[6] Microsoft, „Data binding overview — WPF," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/data/data-binding-overview

[7] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Reading: Addison-Wesley, 1994.

[8] Microsoft, „IValueConverter Interface," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/api/system.windows.data.ivalueconverter

[9] I. Sommerville, *Software Engineering*, 10th ed., Harlow: Pearson, 2015.

[10] The PostgreSQL Global Development Group, „Transaction Isolation — PostgreSQL Documentation," 2024. [Online]. Dostępne: https://www.postgresql.org/docs/current/transaction-iso.html

[11] Microsoft, „Handling concurrency conflicts — EF Core," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/ef/core/saving/concurrency

[12] B. Shneiderman, C. Plaisant, M. Cohen, S. Jacobs, N. Elmqvist, *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, 6th ed., Harlow: Pearson, 2016.

[13] Microsoft, „Screen Class," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/api/system.windows.forms.screen

[14] QuestPDF Contributors, „QuestPDF — Open Source .NET Library for PDF Document Generation," 2024. [Online]. Dostępne: https://www.questpdf.com/

[15] ISO/IEC 18004:2015, *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*, Geneva: ISO, 2015.

[16] R. Schulz, „QRCoder — QR code generator for .NET," GitHub, 2024. [Online]. Dostępne: https://github.com/codebude/QRCoder

[17] J. Klensin, „Simple Mail Transfer Protocol," RFC 5321, IETF, 2008. [Online]. Dostępne: https://datatracker.ietf.org/doc/html/rfc5321

[18] Microsoft, „SmtpClient Class," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/api/system.net.mail.smtpclient

[19] J. Nielsen, *Usability Engineering*, San Francisco: Morgan Kaufmann, 1993.

[20] Microsoft, „3D Graphics Overview — WPF," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/graphics-multimedia/3-d-graphics-overview

[21] Microsoft, „Hit Testing in the Visual Layer — WPF," Microsoft Learn, 2024. [Online]. Dostępne: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/graphics-multimedia/hit-testing-in-the-visual-layer